

ANSIBLE — 2.5-HOUR HANDS-ON LAB

Student Step-by-Step Instructions | macOS + Windows (WSL2)

Key rule: Windows students run Ansible inside Ubuntu/WSL2; macOS students run Ansible directly in Terminal.

The lab covers inventory, hosts/groups, YAML, ad-hoc commands, modules, playbooks, variables, package/service/file management, command vs shell, idempotency, and roles.

1. Windows — install WSL2 + Ubuntu

- Open PowerShell as Administrator.
- Run **wsl --status**.
- Install if needed: **wsl --install -d Ubuntu**.
- Restart Windows if requested.
- Launch Ubuntu from Start menu and create the Linux username/password.
- From PowerShell verify: **wsl -l -v**. Ubuntu should be VERSION 2.
- If install hangs at 0%, try **wsl --install --web-download -d Ubuntu**.

2. Windows — install Ansible inside Ubuntu

```
sudo apt update
sudo apt install -y python3 python3-pip pipx
pipx ensurepath
```

Close/reopen Ubuntu, then:

```
python3 --version
pipx --version
pipx install --include-deps ansible
ansible --version
```

If Ansible is not found: **export PATH="\$HOME/.local/bin:\$PATH"** and retry. Current Ansible releases require Python 3.12+ on the control node.

3. Windows — systemd fallback

Modern Ubuntu WSL normally has systemd. Check:

```
systemctl is-system-running
```

If service management later fails and systemd is unavailable:

```
sudo nano /etc/wsl.conf
```

Add:

```
[boot]
systemd=true
```

Save/exit nano, then in PowerShell:

```
wsl --shutdown
```

Reopen Ubuntu and check systemd again.

4. macOS — install Ansible

```
sw_vers
uname -m
python3 --version
brew --version
```

If Homebrew is missing, install it using the official Homebrew installer and follow the printed **brew shellenv** instructions. Then:

```
brew install python pipx
pipx ensurepath
```

Close/reopen Terminal:

```
pipx install --include-deps ansible
ansible --version
```

If Ansible is not found: **export PATH="\$HOME/.local/bin:\$PATH"** and retry.

5. Get and enter the supplied lab files

Extract the supplied ZIP.

Windows/WSL example:

```
cd /mnt/c/Users/YOUR_WINDOWS_USERNAME/Downloads/ansible_2_5_hour_lab/common
ls
```

macOS example:

```
cd ~/Downloads/ansible_2_5_hour_lab/common
ls
```

Install the required collection:

```
ansible-galaxy collection install -r requirements.yml
ansible-galaxy collection list | grep community.general
```

6. Inventory, hosts and groups

```
ansible-inventory -i inventory.ini --graph
ansible-inventory -i inventory.ini --list
```

The supplied inventory contains **localhost** as a host, **webserver**s as a group, and **development** as a parent group. Local execution avoids requiring a second VM/server.

7. Ad-hoc commands

```
ansible webserver -i inventory.ini -m ansible.builtin.ping
ansible webserver -i inventory.ini -m ansible.builtin.command -a "uname -a"
ansible webserver -i inventory.ini -m ansible.builtin.shell -a "echo Hello && whoami"
ansible webserver -i inventory.ini -m ansible.builtin.file -a "path=/tmp/ansible-demo.txt state=touch"
ls -l /tmp/ansible-demo.txt
ansible webserver -i inventory.ini -m ansible.builtin.file -a "path=/tmp/ansible-demo.txt state=absent"
```

Teaching point: command is preferred for a simple command; shell is used when shell operators/features are required.

8. First playbook

```
ansible-playbook -i inventory.ini 02_first_playbook.yml --syntax-check
ansible-playbook -i inventory.ini 02_first_playbook.yml
cat /tmp/ansible-first-playbook.txt
```

9. Variables + idempotency

```
ansible-playbook -i inventory.ini 03_variables.yml --syntax-check
ansible-playbook -i inventory.ini 03_variables.yml
cat /tmp/ansible-variable-demo.txt
ansible-playbook -i inventory.ini 03_variables.yml
```

Run the same playbook twice. Explain **ok** vs **changed** as evidence of desired-state/idempotent behavior.

10. Role — package + service + file copy

```
ansible-playbook -i inventory.ini 04_role_demo.yml --syntax-check
ansible-playbook -i inventory.ini 04_role_demo.yml
```

Ubuntu/WSL uses apt + service + /var/www/html. macOS uses Homebrew + Homebrew services + the Homebrew Nginx document root.

11. Verify the result

macOS: open **http://127.0.0.1:8080**.

Windows/Ubuntu WSL: open **http://127.0.0.1**.

```
curl -I http://127.0.0.1:8080 # macOS
curl -I http://127.0.0.1 # Windows/WSL
```

12. Inspect the role

```
find roles/webserver -maxdepth 3 -type f | sort
cat roles/webserver/tasks/main.yml
```

13. Optional role-generation demo

```
mkdir -p /tmp/ansible-role-demo
cd /tmp/ansible-role-demo
ansible-galaxy role init classroom_demo
find classroom_demo -maxdepth 2 -type f | sort
```

14. Fast troubleshooting

- **ansible not found:** run `pipx ensurepath`, restart shell, or export `PATH="$HOME/.local/bin:$PATH"`.
- **community.general not found:** `ansible-galaxy collection install -r requirements.yml`.
- **No hosts matched:** check group name and always use `-i inventory.ini`.
- **YAML error:** run `--syntax-check`; check indentation and tabs.
- **WSL service/systemd error:** check `systemctl is-system-running`; enable `systemd` via `/etc/wsl.conf` if needed.
- **Nginx browser failure on macOS:** test port 8080 with `curl`; on Ubuntu/WSL test port 80.
- **Permission denied on Ubuntu:** the role uses `become`; enter the Linux `sudo` password when prompted.

15. Final checklist

- Ansible version
- Inventory graph
- Ping/pong
- command
- shell
- file
- first playbook
- variables
- idempotency
- role
- Nginx running
- browser page

16. Official validation notes

The setup was checked against current Ansible, Microsoft WSL, and Homebrew documentation available in September 2026. Ansible's current guidance says Windows is not a native control node; WSL is the practical route. Current `ansible-core 2.20`-era releases require Python 3.12+ on the control node. The role uses `community.general` for macOS Homebrew package/service management.